



DECEMBER 22, 2025

EVADING IDS, FIREWALLS, AND HONEYPOTS

MODULE 12

VYANKATESH ANDIL



TABLE OF CONTENTS

Contents

1. Introduction to evading ids, firewalls, and honeypots	1
1.1 Introduction	1
1.2 What is Evasion in Cybersecurity?	1
1.3 Importance of Studying Evasion Techniques	1
1.4 Overview of Security Controls	2
1.4.1 Intrusion Detection Systems (IDS)	2
1.4.2 Firewalls.....	2
1.4.3 Honeypots	2
1.5 Why Evasion is Possible	3
1.6 Ethical Hacking Perspective	3
1.7 Legal and Ethical Considerations	3
1.8 Scope of This Module.....	4
1.9 Learning Objectives	4
1.10 Summary of Part 1	4
2. Intrusion detection systems (ids): concepts and architecture	5
2.1 Introduction to Intrusion Detection Systems.....	5
2.2 Objectives of IDS	5
2.3 Types of Intrusion Detection Systems	5
2.3.1 Network-Based IDS (NIDS)	6
2.3.2 Host-Based IDS (HIDS).....	6
2.4 IDS Detection Methodologies	6
2.4.1 Signature-Based Detection.....	7
2.4.2 Anomaly-Based Detection.....	7
2.4.3 Stateful Protocol Analysis	7
2.5 IDS Architecture Components.....	8
2.6 IDS Deployment Models	8
2.7 Strengths of IDS.....	8
2.8 Limitations of IDS.....	8

TABLE OF CONTENTS

2.9 Role of IDS in Defense-in-Depth.....	9
2.10 Ethical Hacking Perspective.....	9
2.11 Summary of Part 2.....	9
3. Theoretical concepts of ids evasion.....	10
3.1 Introduction to IDS Evasion	10
3.2 Why IDS Evasion is Possible.....	10
3.3 IDS Blind Spots.....	10
3.4 General Categories of IDS Evasion (Conceptual).....	11
3.5 Traffic Manipulation	11
3.6 Protocol-Level Evasion	11
3.7 Encryption-Based Evasion	12
3.8 Timing-Based Evasion	12
3.9 Resource Exhaustion and Evasion	12
3.10 Obfuscation and Encoding (High-Level)	13
3.11 False Negative Exploitation	13
3.12 Detection vs Prevention Gap	13
3.13 Ethical Hacking Perspective.....	14
3.14 Defensive Implications	14
3.15 Summary of Part 3.....	14
4. Firewalls: concepts, types, and limitations.....	15
4.1 Introduction to Firewalls.....	15
4.2 Objectives of Firewalls.....	15
4.3 Firewall Rule-Based Operation	15
4.4 Types of Firewalls	16
4.4.1 Packet Filtering Firewalls.....	16
4.4.2 Stateful Inspection Firewalls	16
4.4.3 Circuit-Level Gateways.....	17
4.4.4 Application-Level Gateways (Proxy Firewalls)	17
4.4.5 Next-Generation Firewalls (NGFW).....	17
4.5 Firewall Deployment Models	18

TABLE OF CONTENTS

4.6 Strengths of Firewalls.....	18
4.7 Limitations of Firewalls.....	18
4.8 Firewall Blind Spots.....	19
4.9 Firewalls vs IDS.....	19
4.10 Role of Firewalls in Defense-in-Depth.....	19
4.11 Ethical Hacking Perspective.....	20
4.12 Summary of Part 4.....	20
5. Theoretical concepts of firewall evasion	21
5.1 Introduction to Firewall Evasion	21
5.2 Why Firewall Evasion is Possible.....	21
5.3 Firewall Trust Model Weaknesses	21
5.4 Categories of Firewall Evasion (Conceptual).....	22
5.5 Port-Based Evasion (High-Level Concept)	22
5.6 Protocol-Based Evasion.....	22
5.7 Application-Layer Evasion	23
5.8 Fragmentation and Packet Manipulation.....	23
5.9 Encrypted Traffic and Firewall Evasion.....	23
5.10 Firewall Rule Misconfiguration Exploitation.....	23
5.11 Firewall Policy Complexity.....	24
5.12 Firewall Performance Constraints	24
5.13 False Sense of Security	24
5.14 Ethical Hacking Perspective.....	25
5.15 Defensive Implications	25
5.16 Summary of Part 5.....	25
6. Honeypots: concepts, types, and purpose	26
6.1 Introduction to Honeypots	26
6.2 Objectives of Honeypots	26
6.3 How Honeypots Work	26
6.4 Types of Honeypots	27
6.4.1 Low-Interaction Honeypots	27

TABLE OF CONTENTS

6.4.2 High-Interaction Honeypots	27
6.4.3 Production Honeypots	27
6.4.4 Research Honeypots	28
6.5 Honeynet Concept.....	28
6.6 Placement of Honeypots.....	28
6.7 Advantages of Honeypots	28
6.8 Limitations of Honeypots	29
6.9 Legal and Ethical Considerations	29
6.10 Honeypots in Defense-in-Depth	29
6.11 Ethical Hacking Perspective.....	30
6.12 Summary of Part 6	30
7. Theoretical concepts of honeypot detection and evasion	31
7.1 Introduction to Honeypot Detection and Evasion.....	31
7.2 Why Attackers Try to Detect Honeypots	31
7.3 Common Characteristics of Honeypots	31
7.4 Behavioral Indicators of Honeypots	32
7.5 Network-Level Indicators (Conceptual)	32
7.6 System-Level Indicators	32
7.7 Service Interaction Characteristics	33
7.8 Time-Based Observation	33
7.9 Fingerprinting and Environment Analysis (High-Level)	33
7.10 Risk-Avoidance Strategy	34
7.11 Limitations of Honeypot Evasion.....	34
7.12 Defensive Countermeasures.....	34
7.13 Ethical Hacking Perspective.....	35
7.14 Role of Honeypots in Threat Intelligence	35
7.15 Defensive Lessons Learned	35
7.16 Summary of Part 7	35
8. Integrated evasion scenarios and defensive strategies	37
8.1 Introduction to Integrated Security Environments	37

TABLE OF CONTENTS

8.2 Defense-in-Depth and Its Importance	37
8.3 Cross-Layer Evasion Concept	37
8.4 Firewall and IDS Interaction	38
8.5 IDS and Honeypot Integration.....	38
8.6 Firewall and Honeypot Integration.....	38
8.7 Common Integrated Evasion Challenges	39
8.8 Centralized Monitoring and Correlation.....	39
8.9 Behavioral and Context-Aware Defense	39
8.10 Role of Automation and Orchestration.....	39
8.11 Encrypted Traffic Strategy	40
8.12 Reducing Evasion Through Configuration Management.....	40
8.13 Ethical Hacking Perspective.....	40
8.14 Strategic Defensive Recommendations	41
8.15 Lessons Learned from Integrated Defense.....	41
8.16 Summary of Part 8.....	41
9. Conclusion and references	42
9.1 Overall Conclusion	42
9.2 Key Takeaways.....	42
9.3 Importance of Layered Security.....	42
9.4 Role of Ethical Hackers.....	43
9.5 Future Scope	43
9.6 Final Statement	43
9.7 References	44
POC's	45
1. Snort	45

1. INTRODUCTION TO EVADING IDS, FIREWALLS, AND HONEYPOTS

1. Introduction to evading ids, firewalls, and honeypots

1.1 INTRODUCTION

Modern organizations deploy multiple security mechanisms such as **Intrusion Detection Systems (IDS)**, **Firewalls**, and **Honeypots** to protect networks and systems from cyberattacks. These controls are designed to **detect, block, and analyze malicious activity**.

However, attackers continuously develop techniques to **bypass, evade, or deceive** these security mechanisms. Understanding evasion techniques is essential for **ethical hackers and defenders** to evaluate the effectiveness of security controls and improve defensive strategies.

This module focuses on the **theoretical concepts of evasion**, detection weaknesses, and defensive implications.

1.2 WHAT IS EVASION IN CYBERSECURITY?

Evasion refers to techniques used by attackers to:

- Avoid detection by security systems
- Bypass filtering and inspection mechanisms
- Conceal malicious activities
- Mislead monitoring tools

Evasion does not necessarily involve breaking security controls but instead **exploiting their limitations**.

1.3 IMPORTANCE OF STUDYING EVASION TECHNIQUES

Understanding evasion techniques is important because:

- Security tools are not foolproof
- Misconfigurations create blind spots
- Attackers exploit protocol and implementation weaknesses
- Defensive systems rely on assumptions that can be bypassed

1. INTRODUCTION TO EVADING IDS, FIREWALLS, AND HONEYPOTS

Ethical hackers use evasion knowledge to **identify gaps**, not to cause harm.

1.4 OVERVIEW OF SECURITY CONTROLS

1.4.1 Intrusion Detection Systems (IDS)

IDS monitor network or system activity to:

- Detect suspicious behavior
- Generate alerts
- Support incident response

IDS can be:

- Network-based (NIDS)
 - Host-based (HIDS)
-

1.4.2 Firewalls

Firewalls act as a **traffic filtering mechanism**, enforcing security rules based on:

- IP addresses
- Ports
- Protocols
- Application behavior

They serve as the **first line of defense** in most networks.

1.4.3 Honeypots

Honeypots are **decoy systems** designed to:

- Attract attackers
 - Study attack behavior
 - Detect unauthorized access
 - Divert attackers from real assets
-

1. INTRODUCTION TO EVADING IDS, FIREWALLS, AND HONEYPOTS

They play a **detection and research role**, not direct prevention.

1.5 WHY EVASION IS POSSIBLE

Evasion is possible due to:

- Encrypted traffic
- High network speeds
- Limited inspection capabilities
- Signature-based detection weaknesses
- Performance constraints
- Legitimate traffic similarity

Security systems must balance **performance, usability, and accuracy**, which attackers exploit.

1.6 ETHICAL HACKING PERSPECTIVE

From an ethical hacking standpoint, evasion techniques are studied to:

- Test IDS and firewall effectiveness
- Identify false negatives
- Improve detection logic
- Strengthen rule sets
- Enhance network visibility

All testing must be **authorized and legal**.

1.7 LEGAL AND ETHICAL CONSIDERATIONS

Studying evasion techniques must follow:

- Organizational policies
- Legal boundaries

1. INTRODUCTION TO EVADING IDS, FIREWALLS, AND HONEYPOTS

- Written authorization
- Ethical guidelines

Unauthorized evasion attempts are illegal and unethical.

1.8 SCOPE OF THIS MODULE

This module will cover:

- IDS evasion concepts
- Firewall evasion theory
- Honeypot detection and avoidance (theoretical)
- Defensive implications
- Countermeasures and best practices

No practical attack execution is included.

1.9 LEARNING OBJECTIVES

By the end of this module, a learner should be able to:

- Understand how attackers bypass security controls
 - Identify weaknesses in detection systems
 - Recommend defensive improvements
 - Evaluate security architecture resilience
-

1.10 SUMMARY OF PART 1

Part 1 introduced the concept of **security evasion**, the purpose of IDS, firewalls, and honeypots, and why evasion techniques exist. This foundation is essential before exploring evasion methods and defenses in detail.

2. INTRUSION DETECTION SYSTEMS (IDS): CONCEPTS AND ARCHITECTURE

2. Intrusion detection systems (ids): concepts and architecture

2.1 INTRODUCTION TO INTRUSION DETECTION SYSTEMS

An **Intrusion Detection System (IDS)** is a security mechanism designed to **monitor, analyze, and detect suspicious or malicious activity** within a network or host. Unlike firewalls, IDS primarily focus on **detection rather than prevention**.

IDS help security teams by:

- Identifying potential attacks
 - Generating alerts
 - Supporting forensic analysis
 - Enhancing situational awareness
-

2.2 OBJECTIVES OF IDS

The primary objectives of an IDS include:

- Detecting unauthorized access attempts
- Identifying policy violations
- Monitoring abnormal system behavior
- Alerting administrators in real time
- Providing logs for incident investigation

IDS act as an **early warning system**.

2.3 TYPES OF INTRUSION DETECTION SYSTEMS

IDS can be classified based on **deployment location** and **detection methodology**.

2. INTRUSION DETECTION SYSTEMS (IDS): CONCEPTS AND ARCHITECTURE

2.3.1 Network-Based IDS (NIDS)

NIDS are deployed at strategic points within a network.

Key characteristics:

- Monitor network traffic
- Analyze packets in real time
- Detect network-level attacks
- Provide centralized monitoring

Limitations:

- Encrypted traffic visibility is limited
 - High-speed networks reduce inspection depth
-

2.3.2 Host-Based IDS (HIDS)

HIDS are installed on individual systems.

Key characteristics:

- Monitor system logs
- Track file integrity
- Observe process activity
- Detect insider threats

Limitations:

- Limited to the host environment
 - Can be disabled if the host is compromised
-

2.4 IDS DETECTION METHODOLOGIES

IDS detect intrusions using different detection techniques.

2. INTRUSION DETECTION SYSTEMS (IDS): CONCEPTS AND ARCHITECTURE

2.4.1 Signature-Based Detection

This method detects attacks by matching patterns against known signatures.

Advantages:

- High accuracy for known attacks
- Low false positives

Limitations:

- Cannot detect unknown or zero-day attacks
 - Requires frequent signature updates
-

2.4.2 Anomaly-Based Detection

Anomaly-based IDS detect deviations from normal behavior.

Advantages:

- Detects unknown attacks
- Learns baseline behavior

Limitations:

- High false positive rate
 - Requires training period
-

2.4.3 Stateful Protocol Analysis

This method analyzes protocol behavior and state transitions.

Advantages:

- Deep protocol understanding
- Detects protocol misuse

Limitations:

- Resource-intensive
 - Complex configuration
-

2. INTRUSION DETECTION SYSTEMS (IDS): CONCEPTS AND ARCHITECTURE

2.5 IDS ARCHITECTURE COMPONENTS

An IDS typically consists of:

- **Sensors** – Capture and analyze data
 - **Analysis Engine** – Applies detection logic
 - **Event Database** – Stores alerts and logs
 - **Management Console** – Provides visualization and control
-

2.6 IDS DEPLOYMENT MODELS

Common deployment models include:

- Inline monitoring
- Passive network taps
- Mirror ports (SPAN)
- Host-level agents

Deployment affects visibility and performance.

2.7 STRENGTHS OF IDS

IDS provide:

- Visibility into attacks
- Detailed alerting
- Attack pattern identification
- Support for compliance

They enhance defensive awareness.

2.8 LIMITATIONS OF IDS

IDS limitations include:

2. INTRUSION DETECTION SYSTEMS (IDS): CONCEPTS AND ARCHITECTURE

- Inability to block attacks (traditional IDS)
- Dependence on accurate signatures
- Performance overhead
- Evasion susceptibility

Understanding limitations explains why evasion is possible.

2.9 ROLE OF IDS IN DEFENSE-IN-DEPTH

IDS complement:

- Firewalls
- Antivirus systems
- Endpoint protection
- SIEM platforms

They are not standalone solutions.

2.10 ETHICAL HACKING PERSPECTIVE

Ethical hackers evaluate IDS by:

- Testing detection accuracy
- Identifying blind spots
- Assessing false positives
- Improving rule effectiveness

This strengthens organizational security.

2.11 SUMMARY OF PART 2

Part 2 explained IDS concepts, types, detection methods, architecture, strengths, and limitations. This understanding is crucial before analyzing **IDS evasion techniques** in later sections

3. THEORETICAL CONCEPTS OF IDS EVASION

3. Theoretical concepts of ids evasion

3.1 INTRODUCTION TO IDS EVASION

IDS evasion refers to techniques that allow malicious activity to **avoid detection by an Intrusion Detection System**. These techniques do not necessarily break the IDS but instead **exploit its design limitations, assumptions, or configurations**.

From a defensive and ethical hacking perspective, understanding IDS evasion helps organizations:

- Improve detection accuracy
 - Reduce false negatives
 - Strengthen monitoring mechanisms
-

3.2 WHY IDS EVASION IS POSSIBLE

IDS evasion is possible due to several inherent challenges:

- High network traffic volumes
- Performance constraints
- Reliance on signatures or predefined rules
- Encrypted communication
- Protocol complexity
- Normal traffic resemblance

IDS must balance **speed, accuracy, and resource usage**, which attackers attempt to exploit.

3.3 IDS BLIND SPOTS

IDS blind spots are areas where malicious activity may go undetected.

Common blind spots include:

- Encrypted traffic
-

3. THEORETICAL CONCEPTS OF IDS EVASION

- Fragmented packets
- Non-standard protocol usage
- Traffic outside monitored segments
- Resource exhaustion scenarios

Recognizing blind spots is critical for defensive planning.

3.4 GENERAL CATEGORIES OF IDS EVASION (CONCEPTUAL)

IDS evasion techniques can be broadly categorized into:

1. Traffic manipulation
2. Protocol abuse
3. Timing-based evasion
4. Resource exhaustion
5. Obfuscation techniques

Each category targets different detection weaknesses.

3.5 TRAFFIC MANIPULATION

Traffic manipulation involves altering how data is transmitted so it appears benign or confusing to the IDS.

Examples (theoretical):

- Packet fragmentation
- Overlapping packets
- Abnormal packet ordering

These techniques exploit differences between how IDS and end systems process traffic.

3.6 PROTOCOL-LEVEL EVASION

Protocol-level evasion abuses ambiguities in protocol standards.

3. THEORETICAL CONCEPTS OF IDS EVASION

Examples include:

- Non-standard protocol usage
- Exploiting optional fields
- Invalid but accepted protocol states

IDS may interpret traffic differently than the target system.

3.7 ENCRYPTION-BASED EVASION

Encrypted traffic limits IDS visibility.

Key points:

- IDS cannot inspect payloads without decryption
- Encrypted tunnels may hide malicious content
- Increased use of HTTPS reduces deep inspection

This is a major modern challenge for IDS systems.

3.8 TIMING-BASED EVASION

Timing-based evasion focuses on **slowing down or spreading activity** to avoid detection thresholds.

Conceptually:

- IDS may miss low-frequency events
- Threshold-based alerts may not trigger
- Long-duration attacks blend into normal traffic

Detection systems must balance sensitivity.

3.9 RESOURCE EXHAUSTION AND EVASION

Attackers may attempt to:

- Overload IDS processing capacity

3. THEORETICAL CONCEPTS OF IDS EVASION

- Force packet drops
- Reduce inspection accuracy

When overwhelmed, IDS may fail open or reduce detection depth.

3.10 OBFUSCATION AND ENCODING (HIGH-LEVEL)

Obfuscation involves disguising malicious patterns.

Examples (theoretical):

- Encoding data differently
- Altering case or format
- Using uncommon representations

Signature-based IDS are particularly affected.

3.11 FALSE NEGATIVE EXPLOITATION

False negatives occur when:

- Malicious activity is not detected
- Traffic appears legitimate
- Rules do not match altered patterns

IDS evasion aims to increase false negatives.

3.12 DETECTION VS PREVENTION GAP

Traditional IDS only:

- Detect and alert
- Do not block traffic

This gap allows attacks to proceed even if detected late.

3. THEORETICAL CONCEPTS OF IDS EVASION

3.13 ETHICAL HACKING PERSPECTIVE

Ethical hackers study IDS evasion to:

- Identify detection gaps
- Improve signature quality
- Tune anomaly detection
- Recommend architectural changes

The goal is **defensive improvement**, not exploitation.

3.14 DEFENSIVE IMPLICATIONS

Understanding evasion highlights the need for:

- Layered security
 - IDS tuning and updates
 - Integration with IPS, WAF, and SIEM
 - Encrypted traffic inspection strategies
-

3.15 SUMMARY OF PART 3

Part 3 explained:

- What IDS evasion is
- Why IDS evasion exists
- High-level categories of evasion techniques
- Defensive and ethical relevance

This prepares us to study **firewalls** next

4. FIREWALLS: CONCEPTS, TYPES, AND LIMITATIONS

4. Firewalls: concepts, types, and limitations

4.1 INTRODUCTION TO FIREWALLS

A **firewall** is a network security device or software that **monitors and controls incoming and outgoing traffic** based on predefined security rules. Firewalls act as a **barrier between trusted and untrusted networks**, typically separating internal networks from external threats.

Unlike IDS, firewalls primarily focus on **prevention rather than detection**.

4.2 OBJECTIVES OF FIREWALLS

The main objectives of firewalls are:

- Block unauthorized access
- Allow legitimate traffic
- Enforce security policies
- Reduce attack surface
- Control network communications

Firewalls form the **first line of defense** in network security.

4.3 FIREWALL RULE-BASED OPERATION

Firewalls operate using:

- Access control rules
- Policy-based filtering
- Allow/deny logic

Rules are evaluated in sequence, and traffic is either:

- Allowed
- Blocked

4. FIREWALLS: CONCEPTS, TYPES, AND LIMITATIONS

- Logged

Improper rule configuration leads to security gaps.

4.4 TYPES OF FIREWALLS

Firewalls are classified based on inspection depth and functionality.

4.4.1 Packet Filtering Firewalls

These firewalls inspect:

- Source and destination IP addresses
- Ports
- Protocols

Advantages:

- Fast processing
- Low resource usage

Limitations:

- No payload inspection
 - Cannot detect application-layer attacks
-

4.4.2 Stateful Inspection Firewalls

Stateful firewalls track:

- Connection states
- Session information

Advantages:

- Better context awareness
- More secure than packet filters

Limitations:

4. FIREWALLS: CONCEPTS, TYPES, AND LIMITATIONS

- Limited application-layer visibility
 - Susceptible to complex evasion techniques
-

4.4.3 Circuit-Level Gateways

These firewalls:

- Monitor TCP handshakes
- Validate session establishment

Advantages:

- Low overhead
- Hides internal network details

Limitations:

- No content inspection
 - Limited attack detection
-

4.4.4 Application-Level Gateways (Proxy Firewalls)

These firewalls operate at the application layer.

Advantages:

- Deep packet inspection
- Strong security control

Limitations:

- Performance overhead
 - Complex configuration
-

4.4.5 Next-Generation Firewalls (NGFW)

NGFWs combine:

- Stateful inspection
-

4. FIREWALLS: CONCEPTS, TYPES, AND LIMITATIONS

- Application awareness
- Intrusion prevention
- User identity awareness

They provide enhanced visibility but are still not immune to evasion.

4.5 FIREWALL DEPLOYMENT MODELS

Firewalls may be deployed as:

- Network perimeter firewalls
- Internal segmentation firewalls
- Host-based firewalls
- Cloud-based firewalls

Deployment impacts effectiveness and visibility.

4.6 STRENGTHS OF FIREWALLS

Firewalls provide:

- Traffic control
- Attack surface reduction
- Policy enforcement
- Centralized management

They are essential security components.

4.7 LIMITATIONS OF FIREWALLS

Despite their strengths, firewalls have limitations:

- Cannot fully inspect encrypted traffic
- Depend on correct rule configuration

4. FIREWALLS: CONCEPTS, TYPES, AND LIMITATIONS

- May allow malicious traffic that appears legitimate
- Limited detection of insider threats

These limitations make firewall evasion possible.

4.8 FIREWALL BLIND SPOTS

Common blind spots include:

- Allowed service ports
- Trusted network zones
- Encrypted tunnels
- Application-layer attacks

Attackers attempt to hide within allowed traffic.

4.9 FIREWALLS VS IDS

Feature	Firewall	IDS
Primary Role	Prevention	Detection
Traffic Blocking	Yes	No (traditional)
Payload Analysis	Limited (depends on type)	Extensive
Alerting	Limited	Detailed

Both must work together.

4.10 ROLE OF FIREWALLS IN DEFENSE-IN-DEPTH

Firewalls complement:

- IDS/IPS
- Endpoint security

4. FIREWALLS: CONCEPTS, TYPES, AND LIMITATIONS

- WAF
- SIEM systems

They are not standalone solutions.

4.11 ETHICAL HACKING PERSPECTIVE

Ethical hackers analyze:

- Firewall rule effectiveness
- Misconfigurations
- Overly permissive policies
- Security gaps

This improves firewall security posture.

4.12 SUMMARY OF PART 4

Part 4 covered:

- Firewall fundamentals
- Types and deployment models
- Strengths and limitations
- Why firewall evasion is possible

This prepares us to study **firewall evasion concepts** next.

5. THEORETICAL CONCEPTS OF FIREWALL EVASION

5. Theoretical concepts of firewall evasion

5.1 INTRODUCTION TO FIREWALL EVASION

Firewall evasion refers to techniques used to **bypass firewall rules and filtering mechanisms** without directly disabling the firewall. These techniques exploit **design limitations, rule misconfigurations, protocol behaviors, and trusted traffic assumptions**.

From a defensive perspective, understanding firewall evasion helps security teams:

- Identify policy gaps
 - Strengthen rule sets
 - Improve traffic inspection
 - Reduce attack surface
-

5.2 WHY FIREWALL EVASION IS POSSIBLE

Firewall evasion is possible due to:

- Overly permissive rules
- Allowed service ports
- Encrypted traffic
- Application-layer complexity
- Performance constraints
- Trust relationships between networks

Firewalls must allow business traffic, which attackers attempt to blend into.

5.3 FIREWALL TRUST MODEL WEAKNESSES

Firewalls often rely on:

- Trusted internal networks
-

5. THEORETICAL CONCEPTS OF FIREWALL EVASION

- Allowed protocols and ports
- Known applications

Attackers exploit this trust by disguising malicious activity as **legitimate traffic**.

5.4 CATEGORIES OF FIREWALL EVASION (CONCEPTUAL)

Firewall evasion techniques can be broadly classified into:

1. Port-based evasion
2. Protocol-based evasion
3. Application-layer evasion
4. Fragmentation and manipulation
5. Encrypted channel abuse

These categories target different firewall inspection layers.

5.5 PORT-BASED EVASION (HIGH-LEVEL CONCEPT)

Port-based evasion relies on the fact that:

- Firewalls allow specific ports (e.g., web traffic)
- Malicious traffic may be hidden within allowed ports

If inspection is shallow, traffic may pass through unnoticed.

5.6 PROTOCOL-BASED EVASION

Protocol-based evasion exploits:

- Protocol ambiguities
- Optional fields
- Non-standard implementations

Firewalls may interpret traffic differently than the destination system, leading to bypass.

5. THEORETICAL CONCEPTS OF FIREWALL EVASION

5.7 APPLICATION-LAYER EVASION

Application-layer evasion focuses on:

- Mimicking legitimate application behavior
- Using standard application formats
- Embedding malicious logic within normal requests

Firewalls with limited application awareness may fail to detect abuse.

5.8 FRAGMENTATION AND PACKET MANIPULATION

Fragmentation techniques involve:

- Splitting data into smaller units
- Altering packet structure

If the firewall does not properly reassemble traffic, malicious content may evade inspection.

5.9 ENCRYPTED TRAFFIC AND FIREWALL EVASION

Encryption reduces firewall visibility.

Key challenges include:

- Inability to inspect payloads
- Reliance on metadata only
- Increased use of secure tunnels

Firewalls must balance privacy and security.

5.10 FIREWALL RULE MISCONFIGURATION EXPLOITATION

Misconfigurations are a major evasion factor:

- Overly broad rules
- Legacy exceptions

5. THEORETICAL CONCEPTS OF FIREWALL EVASION

- Temporary rules left permanent

Attackers exploit these weaknesses to bypass controls.

5.11 FIREWALL POLICY COMPLEXITY

As firewall policies grow:

- Rule conflicts increase
- Visibility decreases
- Errors become more likely

Complexity creates evasion opportunities.

5.12 FIREWALL PERFORMANCE CONSTRAINTS

Firewalls may:

- Reduce inspection depth under load
- Fail open to maintain availability
- Prioritize speed over security

Performance-driven decisions can be exploited.

5.13 FALSE SENSE OF SECURITY

Organizations may:

- Rely solely on firewalls
- Ignore internal threats
- Underestimate application-layer risks

This mindset increases evasion success.

5. THEORETICAL CONCEPTS OF FIREWALL EVASION

5.14 ETHICAL HACKING PERSPECTIVE

Ethical hackers analyze firewall evasion to:

- Identify weak rules
- Test inspection depth
- Validate segmentation
- Improve policy design

The objective is **defensive enhancement**, not exploitation.

5.15 DEFENSIVE IMPLICATIONS

Understanding firewall evasion highlights the need for:

- Least-privilege rules
 - Regular policy audits
 - Deep packet inspection
 - Integration with IDS/IPS
 - Encrypted traffic inspection strategies
-

5.16 SUMMARY OF PART 5

Part 5 explained:

- What firewall evasion is
- Why it occurs
- Conceptual categories of evasion
- Defensive lessons learned

Next, we move to **honeypots**, which attackers also attempt to detect and avoid.

6. HONEYPOTS: CONCEPTS, TYPES, AND PURPOSE

6. Honeypots: concepts, types, and purpose

6.1 INTRODUCTION TO HONEYPOTS

A **honeypot** is a deliberately deployed **decoy system or service** designed to attract attackers. Unlike firewalls and IDS, honeypots are not meant to block attacks but to **observe, analyze, and understand attacker behavior**.

Honeypots appear to be legitimate targets but have **no production value**, making any interaction suspicious by nature.

6.2 OBJECTIVES OF HONEYPOTS

The primary objectives of honeypots include:

- Detecting unauthorized access attempts
- Collecting attack data
- Studying attacker techniques
- Diverting attackers from real assets
- Improving security awareness

Honeypots are powerful **detection and research tools**.

6.3 HOW HONEYPOTS WORK

Honeypots work by:

- Exposing intentionally vulnerable services
- Logging all interactions
- Monitoring attacker actions
- Triggering alerts on access

Since legitimate users should not interact with honeypots, activity is considered malicious by default.

6. HONEYPOTS: CONCEPTS, TYPES, AND PURPOSE

6.4 TYPES OF HONEYPOTS

Honeypots are classified based on **interaction level** and **deployment purpose**.

6.4.1 Low-Interaction Honeypots

Low-interaction honeypots simulate services.

Characteristics:

- Emulate common protocols
- Limited attacker interaction
- Easy to deploy and maintain

Limitations:

- Limited data collection
 - Easily fingerprinted by skilled attackers
-

6.4.2 High-Interaction Honeypots

High-interaction honeypots run real operating systems and applications.

Characteristics:

- Rich attack data collection
- Realistic environment

Limitations:

- Higher risk
 - Complex management
 - Requires strong containment
-

6.4.3 Production Honeypots

Production honeypots are deployed within live environments.

Purpose:

6. HONEYPOTS: CONCEPTS, TYPES, AND PURPOSE

- Detect ongoing attacks
 - Alert administrators
 - Enhance network monitoring
-

6.4.4 Research Honeypots

Research honeypots are used for:

- Studying attack patterns
 - Developing defensive strategies
 - Academic and threat research
-

6.5 HONEYNET CONCEPT

A **honeynet** is a network of honeypots designed to simulate a realistic enterprise environment. It provides broader visibility into attack techniques and lateral movement behavior.

6.6 PLACEMENT OF HONEYPOTS

Honeypots may be placed:

- At network perimeter
- Within internal networks
- In DMZ zones
- In cloud environments

Placement affects visibility and risk.

6.7 ADVANTAGES OF HONEYPOTS

Honeypots offer:

- High signal-to-noise ratio
 - Early attack detection
-

6. HONEYPOTS: CONCEPTS, TYPES, AND PURPOSE

- Insight into attacker behavior
- Minimal false positives

They complement traditional security tools.

6.8 LIMITATIONS OF HONEYPOTS

Honeypots have limitations:

- No prevention capability
- Risk of being compromised
- Limited coverage
- Detectable by advanced attackers

Understanding limitations explains evasion attempts.

6.9 LEGAL AND ETHICAL CONSIDERATIONS

Honeypot deployment must consider:

- Legal authorization
- Privacy concerns
- Data handling regulations
- Network containment

Improper deployment may lead to liability.

6.10 HONEYPOTS IN DEFENSE-IN-DEPTH

Honeypots enhance defense by:

- Providing early warnings
 - Detecting unknown threats
 - Supporting incident response
-

6. HONEYPOTS: CONCEPTS, TYPES, AND PURPOSE

- Improving security intelligence

They should be integrated with SIEM and SOC workflows.

6.11 ETHICAL HACKING PERSPECTIVE

Ethical hackers evaluate:

- Honeypot realism
- Data collection effectiveness
- Detection accuracy
- Containment controls

Their findings help improve honeypot effectiveness.

6.12 SUMMARY OF PART 6

Part 6 covered:

- Honeypot fundamentals
- Types and deployment
- Advantages and limitations
- Role in modern security architectures

Next, we will discuss **honeypot detection and evasion concepts**.

7. THEORETICAL CONCEPTS OF HONEYPOT DETECTION AND EVASION

7. Theoretical concepts of honeypot detection and evasion

7.1 INTRODUCTION TO HONEYPOT DETECTION AND EVASION

Unlike IDS and firewalls, honeypots do not actively block traffic. Instead, they rely on **deception**. Because honeypots are designed to attract attackers, skilled adversaries attempt to **identify whether a target is real or a decoy** before proceeding.

Honeypot evasion refers to techniques attackers use to:

- Detect decoy systems
- Avoid interacting with them
- Prevent exposure of tools and techniques

Understanding this helps defenders build **more realistic and effective honeypots**.

7.2 WHY ATTACKERS TRY TO DETECT HONEYPOTS

Attackers attempt honeypot detection to:

- Avoid being monitored or logged
- Protect their attack techniques
- Prevent attribution or analysis
- Reduce risk of countermeasures

If a honeypot is detected, attackers usually disengage.

7.3 COMMON CHARACTERISTICS OF HONEYPOTS

Attackers look for anomalies such as:

- Lack of normal user activity
- Minimal system interaction history
- Simplified or default configurations

7. THEORETICAL CONCEPTS OF HONEYPOT DETECTION AND EVASION

- Limited service diversity
- Predictable responses

These characteristics may indicate a decoy system.

7.4 BEHAVIORAL INDICATORS OF HONEYPOTS

Behavior-based indicators include:

- Unrealistic uptime patterns
- No legitimate traffic
- Immediate responses to probes
- Uniform system behavior

Real systems usually exhibit organic activity patterns.

7.5 NETWORK-LEVEL INDICATORS (CONCEPTUAL)

From a network perspective, honeypots may show:

- Simplified network paths
- Unusual routing behavior
- Lack of outbound connections
- Restricted network communication

Such patterns may raise suspicion.

7.6 SYSTEM-LEVEL INDICATORS

At the system level, attackers may theoretically observe:

- Limited running processes
- Missing user artifacts
- Unusual system performance

7. THEORETICAL CONCEPTS OF HONEYPOT DETECTION AND EVASION

- Simplified file structures

These may suggest a controlled environment.

7.7 SERVICE INTERACTION CHARACTERISTICS

Honeypots may:

- Simulate services imperfectly
- Respond differently to malformed inputs
- Lack full protocol implementation

Advanced attackers test for such inconsistencies.

7.8 TIME-BASED OBSERVATION

Attackers may observe systems over time to detect:

- Static behavior
- Lack of configuration changes
- Absence of updates or patches
- Predictable response timing

Time-based analysis helps identify decoys.

7.9 FINGERPRINTING AND ENVIRONMENT ANALYSIS (HIGH-LEVEL)

Conceptually, attackers may analyze:

- Virtualization artifacts
- System metadata
- Environmental consistency

These indicators may reveal monitored environments.

7. THEORETICAL CONCEPTS OF HONEYPOT DETECTION AND EVASION

7.10 RISK-AVOIDANCE STRATEGY

Once a honeypot is suspected, attackers may:

- Cease interaction
- Avoid exploitation
- Change attack paths
- Target different systems

This is considered successful honeypot evasion.

7.11 LIMITATIONS OF HONEYPOT EVASION

Honeypot detection is not always reliable because:

- High-interaction honeypots are realistic
- Production honeypots blend with real systems
- Advanced deception techniques exist

False assumptions may cause attackers to miss real targets.

7.12 DEFENSIVE COUNTERMEASURES

Defenders improve honeypot effectiveness by:

- Increasing realism
- Simulating normal user behavior
- Randomizing system responses
- Integrating honeypots into real environments
- Updating configurations regularly

Better deception reduces evasion success.

7. THEORETICAL CONCEPTS OF HONEYPOT DETECTION AND EVASION

7.13 ETHICAL HACKING PERSPECTIVE

Ethical hackers assess:

- Honeypot detectability
- Deception quality
- Data collection accuracy
- Attacker interaction realism

This ensures honeypots provide actionable intelligence.

7.14 ROLE OF HONEYPOTS IN THREAT INTELLIGENCE

Even when detected, honeypots may still:

- Divert attackers
- Delay attacks
- Reveal reconnaissance techniques
- Provide early warning signals

Partial interaction can still be valuable.

7.15 DEFENSIVE LESSONS LEARNED

Key lessons include:

- Deception must be realistic
 - Static honeypots are easily detected
 - Integration improves effectiveness
 - Continuous tuning is essential
-

7.16 SUMMARY OF PART 7

Part 7 covered:

7. THEORETICAL CONCEPTS OF HONEYPOT DETECTION AND EVASION

- Why attackers detect honeypots
- Conceptual detection indicators
- Evasion strategies
- Defensive improvements

Next, we integrate **IDS, firewalls, and honeypots together**.

8. INTEGRATED EVASION SCENARIOS AND DEFENSIVE STRATEGIES

8. Integrated evasion scenarios and defensive strategies

8.1 INTRODUCTION TO INTEGRATED SECURITY ENVIRONMENTS

Modern enterprise networks rarely rely on a single security control. Instead, they deploy a **layered defense architecture** consisting of:

- Firewalls (traffic control)
- IDS/IPS (detection and alerting)
- Honeypots (deception and intelligence)

Attackers therefore do not evade one system in isolation but attempt to **bypass multiple controls simultaneously**. Understanding this interaction is critical for effective defense.

8.2 DEFENSE-IN-DEPTH AND ITS IMPORTANCE

Defense-in-depth is a strategy where multiple security layers protect assets.

Key principles include:

- No single point of failure
- Overlapping controls
- Compensating mechanisms
- Continuous monitoring

Evasion becomes harder as attackers must defeat multiple systems.

8.3 CROSS-LAYER EVASION CONCEPT

In integrated environments, evasion may involve:

- Traffic that passes firewall rules
- Payloads that evade IDS signatures
- Behavior that avoids honeypot interaction

8. INTEGRATED EVASION SCENARIOS AND DEFENSIVE STRATEGIES

Even if one layer fails, others may still detect the activity.

8.4 FIREWALL AND IDS INTERACTION

Firewalls and IDS work together as follows:

- Firewalls filter traffic
- IDS analyze allowed traffic
- IDS alerts trigger firewall rule updates

Evasion risk:

If firewall rules are too permissive, IDS becomes overloaded.

If IDS signatures are weak, malicious traffic passes silently.

8.5 IDS AND HONEYPOT INTEGRATION

IDS and honeypots complement each other by:

- IDS detecting suspicious traffic
- Honeypots capturing attacker behavior
- Shared intelligence improving detection

Defensive advantage:

Honeypots can reveal evasion attempts that IDS misses.

8.6 FIREWALL AND HONEYPOT INTEGRATION

Firewalls support honeypots by:

- Redirecting suspicious traffic
- Isolating honeypot networks
- Containing compromised honeypots

This prevents attackers from reaching real assets.

8. INTEGRATED EVASION SCENARIOS AND DEFENSIVE STRATEGIES

8.7 COMMON INTEGRATED EVASION CHALLENGES

Organizations face challenges such as:

- Inconsistent policies across tools
- Poor visibility into encrypted traffic
- Alert fatigue
- Lack of correlation between systems

Attackers exploit these gaps.

8.8 CENTRALIZED MONITORING AND CORRELATION

Security Information and Event Management (SIEM) systems:

- Correlate firewall, IDS, and honeypot logs
- Detect patterns across layers
- Identify low-and-slow attacks

Correlation reduces evasion success.

8.9 BEHAVIORAL AND CONTEXT-AWARE DEFENSE

Modern defense focuses on:

- User behavior analytics
- Context-based access control
- Continuous verification

Behavioral monitoring helps detect evasion even when signatures fail.

8.10 ROLE OF AUTOMATION AND ORCHESTRATION

Security orchestration improves response by:

- Automatically blocking suspicious traffic

8. INTEGRATED EVASION SCENARIOS AND DEFENSIVE STRATEGIES

- Isolating compromised systems
- Updating detection rules

Automation reduces attacker dwell time.

8.11 ENCRYPTED TRAFFIC STRATEGY

Since encryption limits inspection:

- TLS inspection may be used selectively
- Metadata analysis becomes important
- Endpoint visibility is critical

Balanced approaches maintain privacy and security.

8.12 REDUCING EVASION THROUGH CONFIGURATION MANAGEMENT

Regular reviews help:

- Remove outdated firewall rules
- Update IDS signatures
- Refresh honeypot configurations

Configuration drift creates evasion opportunities.

8.13 ETHICAL HACKING PERSPECTIVE

Ethical hackers test:

- End-to-end security effectiveness
- Cross-tool communication
- Detection consistency
- Incident response readiness

Integrated testing reveals real-world weaknesses.

8. INTEGRATED EVASION SCENARIOS AND DEFENSIVE STRATEGIES

8.14 STRATEGIC DEFENSIVE RECOMMENDATIONS

Best practices include:

- Layered security architecture
- Continuous monitoring
- Regular tuning and updates
- Threat intelligence integration
- User and admin training

No single control is sufficient alone.

8.15 LESSONS LEARNED FROM INTEGRATED DEFENSE

Key lessons:

- Evasion targets gaps, not tools
 - Integration improves detection
 - Visibility is more important than volume of tools
 - Coordination matters more than complexity
-

8.16 SUMMARY OF PART 8

Part 8 explained:

- How attackers conceptually evade multiple controls
- Why integration is critical
- Defensive strategies to reduce evasion success
- Importance of centralized monitoring and automation

We now move to the **final conclusion and references**.

9. CONCLUSION AND REFERENCES

9. Conclusion and references

9.1 OVERALL CONCLUSION

This report presented a comprehensive theoretical study of **Evading IDS, Firewalls, and Honeypots**, focusing on how attackers conceptually bypass security controls and how defenders can strengthen detection and prevention mechanisms.

The discussion demonstrated that evasion is not achieved by breaking security systems directly, but by **exploiting limitations, assumptions, misconfigurations, and performance constraints** inherent in security technologies. IDS, firewalls, and honeypots each serve a critical role, yet none are sufficient when deployed in isolation.

9.2 KEY TAKEAWAYS

The key conclusions drawn from this module include:

- IDS evasion exploits detection limitations and blind spots.
 - Firewall evasion targets rule weaknesses and trusted traffic assumptions.
 - Honeypot evasion focuses on detecting deception and avoiding monitoring.
 - Encrypted traffic significantly reduces visibility for all controls.
 - Misconfiguration and complexity are major enablers of evasion.
 - Integration and correlation across tools are essential for defense.
-

9.3 IMPORTANCE OF LAYERED SECURITY

A major lesson from this study is the importance of **defense-in-depth**. Effective security does not rely on a single tool, but on:

- Properly configured firewalls
 - Tuned IDS/IPS systems
 - Realistic honeypots
-

9. CONCLUSION AND REFERENCES

- Centralized monitoring and correlation
- Continuous improvement and updates

Layered security significantly increases the cost and complexity of evasion for attackers.

9.4 ROLE OF ETHICAL HACKERS

Ethical hackers play a vital role by:

- Testing evasion resistance
- Identifying detection gaps
- Validating configurations
- Improving response readiness

Their work ensures that security controls evolve alongside attack techniques.

9.5 FUTURE SCOPE

As networks evolve, future defensive strategies are expected to focus on:

- AI-driven behavioral analysis
- Zero-trust architectures
- Advanced encrypted traffic inspection
- Deception-based security platforms
- Automated detection and response systems

Understanding evasion techniques will remain essential for modern cybersecurity professionals.

9.6 FINAL STATEMENT

Evading IDS, firewalls, and honeypots remains a persistent challenge in cybersecurity. However, by understanding evasion concepts from a **defensive and ethical perspective**, organizations can design stronger architectures, reduce blind spots, and respond more effectively to threats.

This module reinforces the principle that **security is a continuous process**, not a one-time deployment.

9. CONCLUSION AND REFERENCES

9.7 REFERENCES

Books

1. Stallings, W. – *Network Security Essentials*
2. Easttom, C. – *Computer Security Fundamentals*

Certification Material

3. EC-Council – *CEH v13 Official Courseware, Module 12*

Standards and Frameworks

4. OWASP – *OWASP Top 10*
5. NIST SP 800-61 – *Computer Security Incident Handling Guide*
6. NIST SP 800-94 – *Guide to IDS*
7. ISO/IEC 27001 – *Information Security Management Systems*

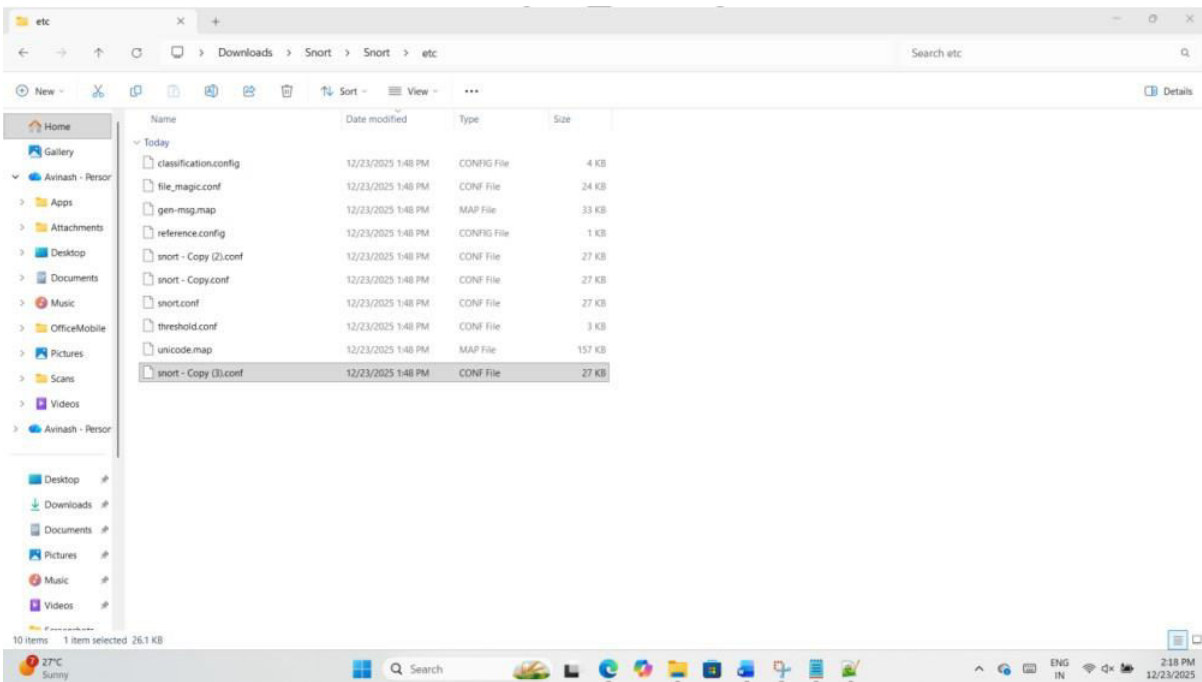
Technical References

8. IETF RFCs – Network and Security Protocol Documentation
9. SANS Institute – Security Whitepapers

POC'S

POC's

1. SNORT



POC'S

```

164 # <var> ::= arbitrary <name>=<value passed to DAQ>
165 # <dir> ::= path as to where to look for DAQ module so's
166
167 # Configure specific UID and GID to run snort as after dropping privs. For more information see snort -h command line options
168 #
169 # config set_gid:
170 # config set_uid:
171
172 # Configure default snaplen. Snort defaults to MTU of in use interface. For more information see README
173 #
174 # config snaplen:
175 #
176
177 # Configure default bpf_file to use for filtering what traffic reaches snort. For more information see snort -h command line options (-F)
178 #
179 # config bpf_file:
180 #
181
182 # Configure default log directory for snort to log to. For more information see snort -h command line options (-l)
183 #
184 config logdir: c:\snort\log
185
186 #####
187 # Step #3: Configure the base detection engine. For more information, see README.decode
188 #
189 #####
190 # Configure PCRE match limitations
191 config pcre_match_limit: 3500
192 config pcre_match_limit_recursion: 1500
193
194 # Configure the detection engine. See the Snort Manual, Configuring Snort - Includes - Config
195 config detection: search-method ac-split search-optimize max-pattern-len 20
196
197 # Configure the event queue. For more information, see README.event_queue
198 config event_queue: max_queue 5 log 5 order_events content_length
199
200 #####
201 ## Configure OTP if it is to be used.
202 ## For more information, see README.OTP
203 #####
204
205 # config match_limit_recursion:
206
207

```

Properties file length: 26,845 lines: 690 Ln: 661 Col: 29 Pos: 25,767 Unix (LF) UTF-8 INS

```

232 #config profile_rules: print all, sort avg_ticks
233 #config profile_preprocs: print all, sort avg_ticks
234
235 #####
236 # Configure protocol aware flushing
237 # For more information see README.stream5
238 #####
239 config pef_max: 16000
240
241 #####
242 # Step #4: Configure dynamic loaded libraries.
243 # For more information, see Snort Manual, Configuring Snort - Dynamic Modules
244 #####
245 # path to dynamic preprocessor libraries
246 dynamicpreprocessor directory c:\snort\lib\snort_dynamicpreprocessor
247
248 # path to base preprocessor engine
249 dynamicengine C:\snort\lib\snort_dynamicengine\af_engine.dll
250
251 # path to dynamic rules libraries
252 #dynamicdetection directory /usr/local/lib/snort_dynamicrules
253
254 #####
255 # Step #5: Configure preprocessors
256 # For more information, see the Snort Manual, Configuring Snort - Preprocessors
257 #####
258 # GTP Control Channel Preprocessor. For more information, see README.OTP
259 # preprocessor gtp: ports ( 2123 3386 2152 )
260
261 # Inline packet normalization. For more information, see README.normalize
262 # Does nothing in IDS mode
263 preprocessor normalize_ip4
264 preprocessor normalize_tcp: ips ecn stream
265 preprocessor normalize_icmp4
266 preprocessor normalize_ip6
267 preprocessor normalize_icmp6
268
269 # Target-based IP defragmentation. For more information, see README.frag3
270 preprocessor frag3_global: max_fragments 65536
271 preprocessor frag3_engine: policy windows detect_anomalies overlap_limit 10 min_fragment_length 100 timeout 180
272
273

```

Properties file length: 26,845 lines: 690 Ln: 661 Col: 29 Pos: 25,767 Unix (LF) UTF-8 INS

POC'S

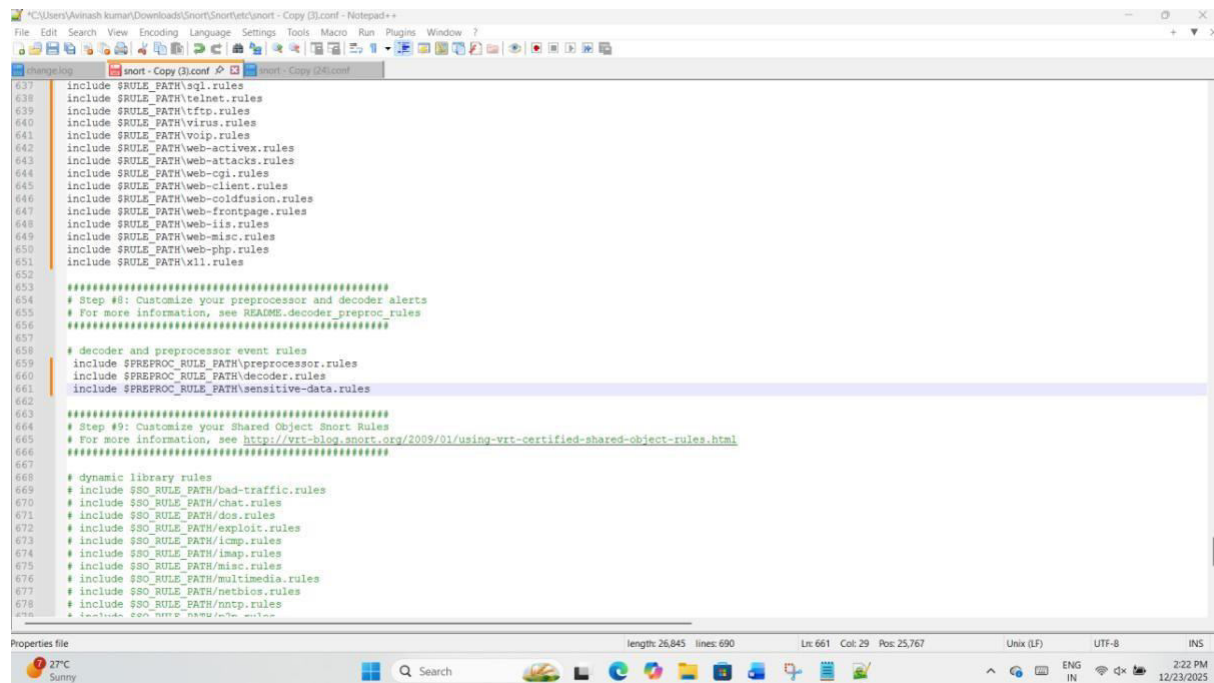
```
487 bitenc_decode_depth 0 \
488 uu_decode_depth 0 \
489
490 # POP preprocessor. For more information see README.pop
491 preprocessor pop: \
492   ports { 110 } \
493   b64_decode_depth 0 \
494   qp_decode_depth 0 \
495   bitenc_decode_depth 0 \
496   uu_decode_depth 0 \
497
498 # Modbus preprocessor. For more information see README.modbus
499 preprocessor modbus: ports { 502 }
500
501 # DNP3 preprocessor. For more information see README.dnp3
502 preprocessor dnp3: ports { 20000 } \
503   memcap 262144 \
504   check_crc \
505
506 # Reputation preprocessor. For more information see README.reputation
507 preprocessor reputation: \
508   memcap 500, \
509   priority whitelist, \
510   nested_ip inner, \
511   whitelist $WHITE_LIST_PATH/white_list.rules, \
512   blacklist $BLACK_LIST_PATH/black_list.rules \
513
514 #####
515 # Step #6: Configure output plugins
516 # For more information, see Snort Manual, Configuring Snort - Output Modules
517 #####
518
519 # unified2
520 # Recommended for most installs
521 # output unified2: filename merged.log, limit 128, nostamp, mpls_event_types, vlan_event_types
522
523 # Additional configuration for specific types of installs
524 # output alert_unified2: filename snort.alert, limit 128, nostamp
525 # output log_unified2: filename snort.log, limit 128, nostamp
526
527 # syslog
528 # output alert_syslog: LOG_AUTH LOG_ALERT
529
530
531
532
533
534 # site specific rules
535 include $RULE_PATH/local.rules
536
537
538 include $RULE_PATH/app-detect.rules
539 include $RULE_PATH/attack-responses.rules
540 include $RULE_PATH/backdoor.rules
541 include $RULE_PATH/bad-traffic.rules
542 include $RULE_PATH/blacklist.rules
543 include $RULE_PATH/botnet-cnc.rules
544 include $RULE_PATH/browser-chrome.rules
545 include $RULE_PATH/browser-firefox.rules
546 include $RULE_PATH/browser-ie.rules
547 include $RULE_PATH/browser-other.rules
548 include $RULE_PATH/browser-plugins.rules
549 include $RULE_PATH/browser-webkit.rules
550 include $RULE_PATH/chat.rules
551 include $RULE_PATH/content-replace.rules
552 include $RULE_PATH/ddos.rules
553 include $RULE_PATH/dns.rules
554 include $RULE_PATH/dos.rules
555 include $RULE_PATH/experimental.rules
556 include $RULE_PATH/exploit-kit.rules
557 include $RULE_PATH/file-executable.rules
558 include $RULE_PATH/file-flash.rules
559 include $RULE_PATH/file-identify.rules
560 include $RULE_PATH/file-image.rules
561 include $RULE_PATH/file-multimedia.rules
562 include $RULE_PATH/file-office.rules
563 include $RULE_PATH/file-pdf.rules
564 include $RULE_PATH/file-other.rules
565 include $RULE_PATH/finger.rules
566 include $RULE_PATH/ftp.rules
567 include $RULE_PATH/icmp-info.rules
568 include $RULE_PATH/icmp.rules
569 include $RULE_PATH/imap.rules
570 include $RULE_PATH/indicator-compromise.rules
571 include $RULE_PATH/indicator-obfuscation.rules
572 include $RULE_PATH/indicator-shellcode.rules
573 include $RULE_PATH/info.rules
574 include $RULE_PATH/malware-backdoor.rules
575 include $RULE_PATH/malware.rules
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

POC'S

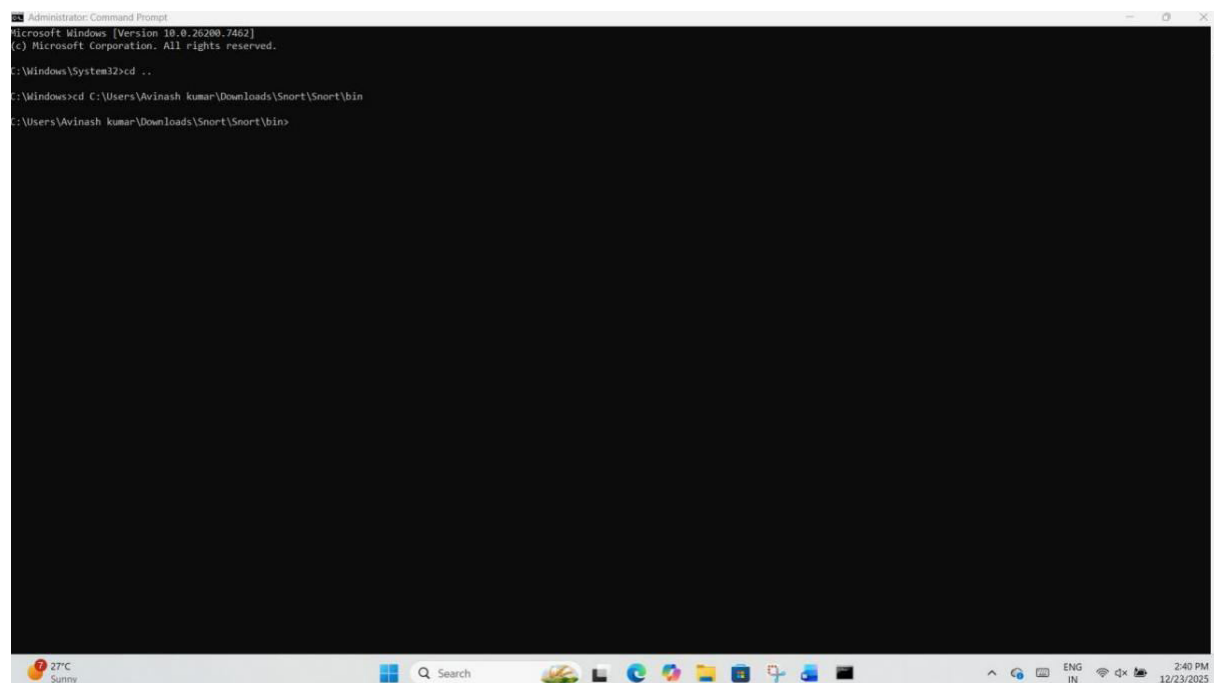
```
*C:\Users\Avinash kumar\Downloads\Snort\Snort\etc\snort - Copy (3).conf - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
snort - Copy (3).conf
571 include $RULE_PATH\file-image.rules
572 include $RULE_PATH\file-multimedia.rules
573 include $RULE_PATH\file-office.rules
574 include $RULE_PATH\file-other.rules
575 include $RULE_PATH\file-pdf.rules
576 include $RULE_PATH\finger.rules
577 include $RULE_PATH\ftp.rules
578 include $RULE_PATH\icmp-info.rules
579 include $RULE_PATH\icmp.rules
580 include $RULE_PATH\imap.rules
581 include $RULE_PATH\indicator-compromise.rules
582 include $RULE_PATH\indicator-obfuscation.rules
583 include $RULE_PATH\indicator-shellcode.rules
584 include $RULE_PATH\info.rules
585 include $RULE_PATH\malware-backdoor.rules
586 include $RULE_PATH\malware-cnc.rules
587 include $RULE_PATH\malware-other.rules
588 include $RULE_PATH\malware-tools.rules
589 include $RULE_PATH\misc.rules
590 include $RULE_PATH\multimedia.rules
591 include $RULE_PATH\mysql.rules
592 include $RULE_PATH\netbios.rules
593 include $RULE_PATH\nntp.rules
594 include $RULE_PATH\oracle.rules
595 include $RULE_PATH\os-linux.rules
596 include $RULE_PATH\os-other.rules
597 include $RULE_PATH\os-solaris.rules
598 include $RULE_PATH\os-windows.rules
599 include $RULE_PATH\other-ids.rules
600 include $RULE_PATH\p2p.rules
601 include $RULE_PATH\phishing-spam.rules
602 include $RULE_PATH\policy-multimedia.rules
603 include $RULE_PATH\policy-other.rules
604 include $RULE_PATH\policy.rules
605 include $RULE_PATH\policy-social.rules
606 include $RULE_PATH\policy-spam.rules
607 include $RULE_PATH\pop2.rules
608 include $RULE_PATH\pop3.rules
609 include $RULE_PATH\protocol-finger.rules
610 include $RULE_PATH\protocol-ftp.rules
611 include $RULE_PATH\protocol-icmp.rules
612 include $RULE_PATH\protocol-imap.rules
613 include $RULE_PATH\protocol-irc.rules
```

```
*C:\Users\Avinash kumar\Downloads\Snort\Snort\etc\snort - Copy (3).conf - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
snort - Copy (3).conf
610 include $RULE_PATH\protocol-ftp.rules
611 include $RULE_PATH\protocol-icmp.rules
612 include $RULE_PATH\protocol-imap.rules
613 include $RULE_PATH\protocol-pop.rules
614 include $RULE_PATH\protocol-services.rules
615 include $RULE_PATH\protocol-voip.rules
616 include $RULE_PATH\psa-aware.rules
617 include $RULE_PATH\psa-other.rules
618 include $RULE_PATH\psa-p2p.rules
619 include $RULE_PATH\psa-toolbars.rules
620 include $RULE_PATH\rpc.rules
621 include $RULE_PATH\services.rules
622 include $RULE_PATH\scada.rules
623 include $RULE_PATH\scan.rules
624 include $RULE_PATH\server-apache.rules
625 include $RULE_PATH\server-iis.rules
626 include $RULE_PATH\server-mail.rules
627 include $RULE_PATH\server-mssql.rules
628 include $RULE_PATH\server-mysql.rules
629 include $RULE_PATH\server-oracle.rules
630 include $RULE_PATH\server-other.rules
631 include $RULE_PATH\server-webapp.rules
632 include $RULE_PATH\shellcode.rules
633 include $RULE_PATH\smtp.rules
634 include $RULE_PATH\snmp.rules
635 include $RULE_PATH\specific-threats.rules
636 include $RULE_PATH\spyware-put.rules
637 include $RULE_PATH\sqli.rules
638 include $RULE_PATH\telnet.rules
639 include $RULE_PATH\tftp.rules
640 include $RULE_PATH\virus.rules
641 include $RULE_PATH\voip.rules
642 include $RULE_PATH\web-activex.rules
643 include $RULE_PATH\web-attacks.rules
644 include $RULE_PATH\web-cgi.rules
645 include $RULE_PATH\web-client.rules
646 include $RULE_PATH\web-coldfusion.rules
647 include $RULE_PATH\web-frontpage.rules
648 include $RULE_PATH\web-iis.rules
649 include $RULE_PATH\web-misc.rules
650 include $RULE_PATH\web-php.rules
651 include $RULE_PATH\x11.rules
```


POC'S



```
637 include $RULE_PATH/sql.rules
638 include $RULE_PATH/telnet.rules
639 include $RULE_PATH/ftp.rules
640 include $RULE_PATH/virus.rules
641 include $RULE_PATH/voip.rules
642 include $RULE_PATH/web-activex.rules
643 include $RULE_PATH/web-attacks.rules
644 include $RULE_PATH/web-cgi.rules
645 include $RULE_PATH/web-client.rules
646 include $RULE_PATH/web-coldfusion.rules
647 include $RULE_PATH/web-frontpage.rules
648 include $RULE_PATH/web-iis.rules
649 include $RULE_PATH/web-misc.rules
650 include $RULE_PATH/web-php.rules
651 include $RULE_PATH/x11.rules
652
653 #####
654 # Step #8: Customize your preprocessor and decoder alerts
655 # For more information, see README-decoder_preproc_rules
656 #####
657
658 # decoder and preprocessor event rules
659 include $PREPROC_RULE_PATH/preprocessor.rules
660 include $PREPROC_RULE_PATH/decoder.rules
661 include $PREPROC_RULE_PATH/sensitive-data.rules
662
663 #####
664 # Step #9: Customize your Shared Object Snort Rules
665 # For more information, see http://vrt-blog.snort.org/2009/01/using-vrt-certified-shared-object-rules.html
666 #####
667
668 # dynamic library rules
669 # include $SO_RULE_PATH/bad-traffic.rules
670 # include $SO_RULE_PATH/chat.rules
671 # include $SO_RULE_PATH/dos.rules
672 # include $SO_RULE_PATH/exploit.rules
673 # include $SO_RULE_PATH/icmp.rules
674 # include $SO_RULE_PATH/imap.rules
675 # include $SO_RULE_PATH/misc.rules
676 # include $SO_RULE_PATH/multimedia.rules
677 # include $SO_RULE_PATH/netbios.rules
678 # include $SO_RULE_PATH/nntp.rules
679 # include $SO_RULE_PATH/ntp.rules
```



```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.26200.7462]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\System32>cd ..
C:\Windows>cd C:\Users\Avinash kumar\Downloads\Snort\Snort\bin
C:\Users\Avinash kumar\Downloads\Snort\Snort\bin>
```


POC'S

```
Administrator: Command Prompt
C:\Users\Avinash kumar\Downloads\Snort\Snort\bin>snort.exe -V

o*~
****
-> Snort! <*-
Version 2.9.20-WIN64 GRE (Build 82)
By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
Copyright (C) 2014-2022 Cisco and/or its affiliates. All rights reserved.
Copyright (C) 1998-2013 Sourcefire, Inc., et al.
Using PCRE version: 8.10 2010-06-25
Using ZLIB version: 1.2.11

C:\Users\Avinash kumar\Downloads\Snort\Snort\bin>snort.exe -w
snort.exe: option requires an argument -- w

o*~
****
-> Snort! <*-
Version 2.9.20-WIN64 GRE (Build 82)
By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
Copyright (C) 2014-2022 Cisco and/or its affiliates. All rights reserved.
Copyright (C) 1998-2013 Sourcefire, Inc., et al.
Using PCRE version: 8.10 2010-06-25
Using ZLIB version: 1.2.11

USAGE: snort.exe [-options] <filter options>
snort.exe /SERVICE /INSTALL [-options] <filter options>
snort.exe /SERVICE /UNINSTALL
snort.exe /SERVICE /SHOW

Options:
-A Set alert mode: fast, full, console, test or none (alert file alerts only)
-b Log packets in tcpdump format (much faster!)
-B cmask Obfuscated IP addresses in alerts and packet dumps using CIDR mask
-c cmask Use Rules file cmask
-C Print out payloads with character data only (no hex)
-d Dump the Application Layer
-e Display the second layer header info
-f Log alert messages to NT Eventlog. (Win32 only)
-f Turn off fflush() calls after binary log writes
-f chpf Read BPF filters from file chpf
-G <uid> Log Identifier (to uniquely id events for multiple snorts)
-h <hn> Set home network = <hn>
    (for use with -l or -B, does NOT change $HOME_NET in IDS mode)
-H Make hash tables deterministic.
-i <if> Listen on interface <if>
-I Add Interface name to alert output
-k cmask Checksum mode (all,noip,notcp,noudp,noicmp,none)
-K cmask Logging mode (pcap[default],ascii,none)
-l <ld> Log to directory <ld>
-L <file> Log to this tcpdump file
-n <cnt> Exit after receiving <cnt> packets
-N Turn off logging (alerts still work)
```

```
Administrator: Command Prompt
C:\Users\Avinash kumar\Downloads\Snort\Snort\bin>snort.exe -W

o*~
****
-> Snort! <*-
Version 2.9.20-WIN64 GRE (Build 82)
By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
Copyright (C) 2014-2022 Cisco and/or its affiliates. All rights reserved.
Copyright (C) 1998-2013 Sourcefire, Inc., et al.
Using PCRE version: 8.10 2010-06-25
Using ZLIB version: 1.2.11

Index Physical Address IP Address Device Name Description
-----
1 00:00:00:00:00:00 disabled \Device\NPF_{92645868-C53F-43E8-B464-7C4A065F83CA} WAN Miniport (Network Monitor)
2 00:00:00:00:00:00 disabled \Device\NPF_{71C78022-C9A2-46AA-A0E0-021EEB13802E} WAN Miniport (IPv6)
3 00:00:00:00:00:00 disabled \Device\NPF_{B11506EF-16A3-4C3A-8AAE-4A61E6C21973} WAN Miniport (IP)
4 EE:C7:CF:A0:A8:D6 192.168.1.35 \Device\NPF_{8FF062CE-30B3-4698-9AAC-B76D0001E963} Realtek RTL8852BE-V5 WiFi 6 802.11ax PCIe Adapter
5 EE:C7:CF:A0:A8:D6 169.254.36.54 \Device\NPF_{24824AB6-5E54-44D1-95E6-DE1AA04FA503} Microsoft Wi-Fi Direct Virtual Adapter #2
6 EA:C7:CF:A0:A8:D6 169.254.121.210 \Device\NPF_{88F951B4-8C1D-45B4-AFA3-F50B118F2F65} Microsoft Wi-Fi Direct Virtual Adapter
7 00:1F:3A:37:C1:A3 169.254.122.221 \Device\NPF_{3A37C143-0D09-4F24-A093-1D6A8AC56887} TAP-Windows Adapter V9
8 0A:00:27:00:00:0C 192.168.56.1 \Device\NPF_{8571554C-6033-414D-9C54-2F458C659CDE} VirtualBox Host-Only Ethernet Adapter
9 00:00:00:00:00:00 0000:0000:0000:0000:0000:0000 \Device\NPF_{loopback} Adapter for loopback traffic capture

C:\Users\Avinash kumar\Downloads\Snort\Snort\bin>
```

POC'S

```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.26200.7462]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\System32>cd ..
C:\Windows>cd \snort
C:\snort>cd bin
C:\snort\bin>snort -i 2 -c "C:\snort\etc\snort - Copy (2).conf"
Running in IDS mode

--== Initializing Snort ==--
initializing Output Plugins!
initializing Preprocessors!
initializing Plug-ins!
Parsing Rules file "C:\snort\etc\snort - Copy (2).conf"
PortVar 'HTTP_PORTS' defined : [ 80:81 311 283 591 593 901 1220 1414 1741 1830 2301 2381 2809 3037 3128 3702 4343 4848 5250 6988 7000:7001 7144:7145 7510 7777 7779 8000 8008 8014 8028 8080 8085 8088 8090 8118 8123 8180:8181 8241 8280 8300 8800 8888 8899 9000 9060 9080 9090:9091 9441 9999 11371 14443:14444 41080 50002 55555 ]
PortVar 'SHELLCODE_PORTS' defined : [ 0:79 81:65535 ]
PortVar 'ORACLE_PORTS' defined : [ 1024:65535 ]
PortVar 'SSH_PORTS' defined : [ 22 ]
PortVar 'FTP_PORTS' defined : [ 21 2100 3535 ]
PortVar 'SIP_PORTS' defined : [ 5060:5061 5060 ]
PortVar 'FILE_DATA_PORTS' defined : [ 80:81 110 143 311 383 591 593 901 1220 1414 1741 1830 2301 2381 2809 3037 3128 3702 4343 4848 5250 6988 7000:7001 7144:7145 7510 7777 7779 8000 8008 8014 8028 8080 8085 8088 8090 8118 8123 8180:8181 8241 8280 8300 8800 8888 8899 9000 9060 9080 9090:9091 9441 9999 11371 14443:14444 41080 50002 55555 ]
PortVar 'GTP_PORTS' defined : [ 2123 2152 3386 ]
Detection:
  Search-Method = AC-Full-Q
  Split Any/Any group = enabled
  Search-Method-Optimizations = enabled
  Maximum pattern length = 20
  Agg'd Packet Limit = 256
Loading dynamic engine C:\Snort\lib\snort_dynamicengine\sf_engine.dll... done
Loading all dynamic preprocessor libs from C:\Snort\lib\snort_dynamicpreprocessor\...
Loading dynamic preprocessor library C:\Snort\lib\snort_dynamicpreprocessor\sf_dce2.dll... done
Loading dynamic preprocessor library C:\Snort\lib\snort_dynamicpreprocessor\sf_dce3.dll... done
Loading dynamic preprocessor library C:\Snort\lib\snort_dynamicpreprocessor\sf_dns.dll... done
Loading dynamic preprocessor library C:\Snort\lib\snort_dynamicpreprocessor\sf_ftptelnet.dll... done
Loading dynamic preprocessor library C:\Snort\lib\snort_dynamicpreprocessor\sf_gtp.dll... done
Loading dynamic preprocessor library C:\Snort\lib\snort_dynamicpreprocessor\sf_imap.dll... done
Loading dynamic preprocessor library C:\Snort\lib\snort_dynamicpreprocessor\sf_modbus.dll... done
Loading dynamic preprocessor library C:\Snort\lib\snort_dynamicpreprocessor\sf_pop.dll... done
Loading dynamic preprocessor library C:\Snort\lib\snort_dynamicpreprocessor\sf_reputation.dll... done
Loading dynamic preprocessor library C:\Snort\lib\snort_dynamicpreprocessor\sf_sdf.dll... done
Loading dynamic preprocessor library C:\Snort\lib\snort_dynamicpreprocessor\sf_sip.dll... done
Loading dynamic preprocessor library C:\Snort\lib\snort_dynamicpreprocessor\sf_smtp.dll... done
Loading dynamic preprocessor library C:\Snort\lib\snort_dynamicpreprocessor\sf_ssh.dll... done
Loading dynamic preprocessor library C:\Snort\lib\snort_dynamicpreprocessor\sf_ssl.dll... done
```

```
Administrator: Command Prompt
Finished Loading all dynamic preprocessor libs from C:\Snort\lib\snort_dynamicpreprocessor\
Log directory = C:\Snort\log
WARNING: ip4 normalizations disabled because not inline.
WARNING: tcp normalizations disabled because not inline.
WARNING: icmp4 normalizations disabled because not inline.
WARNING: ip6 normalizations disabled because not inline.
WARNING: icmp6 normalizations disabled because not inline.
Frag3 global config:
  Max frags: 65536
  Fragment memory cap: 4194304 bytes
Frag3 engine config:
  Bound Address: default
  Target-based policy: WINDOWS
  Fragment timeout: 180 seconds
  Fragment min.ttl: 1
  Fragment Anomalies: Alert
  Overlap Limit: 10
  Min fragment length: 100
  Max Expected Streams: 768
Stream global config:
  Track TCP sessions: ACTIVE
  Max TCP sessions: 262144
  TCP cache pruning timeout: 30 seconds
  TCP cache nominal timeout: 3600 seconds
  Memcap (for reassembly packet storage): 8388608
  Track UDP sessions: ACTIVE
  Max UDP sessions: 131072
  UDP cache pruning timeout: 30 seconds
  UDP cache nominal timeout: 180 seconds
  Track ICMP sessions: INACTIVE
  Track IP sessions: INACTIVE
  Log info if session memory consumption exceeds 1048576
  Send up to 2 active responses
  Wait at least 5 seconds between responses
  Protocol Aware Flushing: ACTIVE
  Maximum Flush Point: 16000
Stream TCP Policy config:
  Bound Address: default
  Reassembly Policy: WINDOWS
  Timeout: 180 seconds
  Limit on TCP Overlaps: 10
  Maximum number of bytes to queue per session: 1048576
  Maximum number of segs to queue per session: 2621
Options:
  Require 3-Way Handshake: YES
  3-Way Handshake Timeout: 180
  Detect Anomalies: YES
  Reassembly Ports:
  21 client (Footprint)
```

POC'S

```
Administrator: Command Prompt
Maximum number of segs to queue per session: 2621
Options:
  Require 3-Way Handshake: YES
  3-Way Handshake Timeout: 180
  Detect Anomalies: YES
Reassembly Ports:
  21 client (Footprint)
  22 client (Footprint)
  23 client (Footprint)
  25 client (Footprint)
  42 client (Footprint)
  53 client (Footprint)
  79 client (Footprint)
  80 client (Footprint) server (Footprint)
  81 client (Footprint) server (Footprint)
  109 client (Footprint)
  110 client (Footprint)
  111 client (Footprint)
  113 client (Footprint)
  119 client (Footprint)
  135 client (Footprint)
  136 client (Footprint)
  137 client (Footprint)
  139 client (Footprint)
  143 client (Footprint)
  161 client (Footprint)
additional ports configured but not printed.
Stream UDP Policy config:
  Timeout: 180 seconds
Httpinspect Config:
GLOBAL CONFIG:
  Detect Proxy Usage: NO
  IIS Unicode Map Filename: C:\vsnort\etc\unicode.map
  IIS Unicode Map Codepage: 1252
  Memsap used for logging URI and Hostname: 150994944
  Max Gzip Memory: 838860
  Max Gzip Sessions: 2016
  Gzip Compress Depth: 65535
  Gzip Decompress Depth: 65535
  Normalize Random Nulls in Text: NO
DEFAULT SERVER CONFIG:
  Server profile: All
  Ports (PAS): 80 81 311 383 501 593 901 1220 1414 1741 1830 2301 2381 2809 3037 3128 3782 4343 4848 5250 6988 7000 7001 7144 7145 7510 7777 7779 8000 8008 8014 8028 8080 8085 8088 8090 8118 8123 8180 8181 8243 8280 8380 8800 8888 8990 9000 9060 9080 9090 9091 9443 9999 11371 14443 14444 41000 50002 55555
  Server Flow Depth: 0
  Client Flow Depth: 0
  Max Chunk Length: 500000
  Small Chunk Length Evasion: chunk size <= 10, threshold >= 5 times
  Max Header Field length: 750
```

```
Administrator: Command Prompt
Reputation total entries loaded: 0, invalid: 0, re-defined: 0
Memsap: 500 (Default) M bytes
Scan local network: DISABLED (Default)
Reputation priority: whitelist(Default)
Nested IP: inner (Default)
White action: unblack (Default)
Shared memory is not supported.

*****
Initializing rule chains...
11088 Snort rules read
  10644 detection rules
  153 decoder rules
  291 preprocessor rules
11088 Option Chains linked into 331 Chain Headers
*****

-----[Rule Port Counts]-----
|      tcp    udp    icmp    ip
|  src  3831    23      0      0
|  dst  6442    77      0      0
|  any   712     4      3      0
|  nc    452     0      0      0
|  s+d    4      2      0      0
-----

-----[detection-filter-config]-----
| memory-cap : 1048576 bytes
-----[detection-filter-rules]-----
-----

-----[rate-filter-config]-----
| memory-cap : 1048576 bytes
-----[rate-filter-rules]-----
| none
-----

-----[event-filter-config]-----
| memory-cap : 1048576 bytes
-----[event-filter-global]-----
-----[event-filter-local]-----
| none
-----[suppression]-----
| none
-----

Rule application order: pass->drop->sdrop->reject->alert->log
Verifying Preprocessor Configurations!
WARNING: flowbits key 'Bandook135_detection' is set but not ever checked.
```

POC'S

```
Administrator: Command Prompt
WARNING: flowbits key "file.crx" is set but not ever checked.
399 out of 1024 flowbits in use.

Port Based Pattern Matching Memory ]
-- [ Aho-Corasick Summary ] -----
Storage Format      : Full-Q
Finite Automaton    : DFA
Alphabet Size       : 256 Chars
Sizeof State        : Variable (1,2,4 bytes)
Instances           : 227
  1 byte states : 214
  2 byte states : 11
  4 byte states : 2
Characters          : 229293
States              : 181676
Transitions         : 31755877
State Density       : 68.3%
Patterns            : 20772
Match States        : 11066
Memory (MB)         : 162.28
  Patterns         : 1.26
  Match Lists      : 2.85
DFA
  1 byte states : 1.25
  2 byte states : 19.06
  4 byte states : 137.47

-----
Number of patterns truncated to 20 bytes: 651 ]
Scap DAQ configured to passive.
The DAQ version does not support reload.
Acquiring network traffic from "\Device\NPF_{71C7F022-C9A2-46AA-A0E0-D21E0D13B82E}".
Decoding Ethernet

--- Initialization Complete ---

o*~--> Snort! <~
o*~--> Version 2.9.20-MING64 GRE (Build 82)
o*~--> By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
o*~--> Copyright (C) 2014-2022 Cisco and/or its affiliates. All rights reserved.
o*~--> Copyright (C) 1998-2013 Sourcefire, Inc., et al.
o*~--> Using PCRE version: 8.10 2010-06-25
o*~--> Using ZLIB version: 1.2.11

Rules Engine: SF-SNORT-DETECTION-ENGINE Version 3.2 <Build 1>
Preprocessor Object: SF-SSLPP Version 1.1 <Build 4>
Preprocessor Object: SF-SSH Version 1.1 <Build 3>
Preprocessor Object: SF-SMTP Version 1.1 <Build 9>
Preprocessor Object: SF-SIP Version 1.1 <Build 1>
Preprocessor Object: SF-SOF Version 1.1 <Build 1>
```

```
Administrator: Command Prompt
Preprocessor Object: SF-OCERPC2 Version 1.0 <Build 3>
commencing packet processing (pid=6180)
** Caught Int-Signal
=====
run time for packet processing was 25.204000 seconds
snort processed 0 packets.
snort ran for 0 days 0 hours 0 minutes 25 seconds
Pkts/sec:
=====
Packet I/O Totals:
  Received: 0
  Analyzed: 0 ( 0.000%)
  Dropped: 0 ( 0.000%)
  Filtered: 0 ( 0.000%)
  Outstanding: 0 ( 0.000%)
  Injected: 0
=====
Breakdown by protocol (includes rebuilt packets):
  Eth: 0 ( 0.000%)
  VLAN: 0 ( 0.000%)
  IP4: 0 ( 0.000%)
  Frag: 0 ( 0.000%)
  ICMP: 0 ( 0.000%)
  UDP: 0 ( 0.000%)
  TCP: 0 ( 0.000%)
  IP6: 0 ( 0.000%)
  IP6 Ext: 0 ( 0.000%)
  IP6 Opts: 0 ( 0.000%)
  Frag6: 0 ( 0.000%)
  ICMP6: 0 ( 0.000%)
  UDP6: 0 ( 0.000%)
  TCP6: 0 ( 0.000%)
  Teredo: 0 ( 0.000%)
  ICMP-IP: 0 ( 0.000%)
  EAPOL: 0 ( 0.000%)
  IP4/IP4: 0 ( 0.000%)
  IP4/IP6: 0 ( 0.000%)
  IP6/IP4: 0 ( 0.000%)
  IP6/IP6: 0 ( 0.000%)
  GRE: 0 ( 0.000%)
  GRE Eth: 0 ( 0.000%)
  GRE VLAN: 0 ( 0.000%)
  GRE IP4: 0 ( 0.000%)
  GRE IP6: 0 ( 0.000%)
  GRE IP6 Ext: 0 ( 0.000%)
  GRE PPTP: 0 ( 0.000%)
  GRE ARP: 0 ( 0.000%)
  GRE IPX: 0 ( 0.000%)
  GRE Loop: 0 ( 0.000%)
```

POC'S

```
Select Administrator: Command Prompt

C:\snort\bin>snort -i 2 -c "C:\snort\etc\snort - Copy (2).conf" -T
Running in test mode

--- Initializing Snort ---
Initializing Output Plugins!
Initializing Preprocessors!
Initializing Plug-ins!
Parsing Rules file "C:\snort\etc\snort - Copy (2).conf"
PortVar 'HTTP_PORTS' defined : [ 80:81 311 383 593 901 1220 1414 1741 1830 2381 2889 3037 3128 3702 4343 4848 5250 6988 7000:7001 7144:7145 7510 7777 7779 8000 8008 8014 8028 8080 8085 8088 8090 8118 8123 8180:8181 8243 8280 8300 8800 8888 8899 9000 9060 9080 9090:9091 9443 9999 11371 34443:34444 41080 50002 55555 ]
PortVar 'SHELLCODE_PORTS' defined : [ 0:79 81:65535 ]
PortVar 'ORACLE_PORTS' defined : [ 1024:65535 ]
PortVar 'SSH_PORTS' defined : [ 22 ]
PortVar 'FTP_PORTS' defined : [ 21 2100 3535 ]
PortVar 'SIP_PORTS' defined : [ 5060:5061 5600 ]
PortVar 'FILE_DATA_PORTS' defined : [ 80:81 118 143 311 383 593 901 1220 1414 1741 1830 2381 2889 3037 3128 3702 4343 4848 5250 6988 7000:7001 7144:7145 7510 7777 7779 8000 8008 8014 8028 8080 8085 8088 8090 8118 8123 8180:8181 8243 8280 8300 8800 8888 8899 9000 9060 9080 9090:9091 9443 9999 11371 34443:34444 41080 50002 55555 ]
PortVar 'GTP_PORTS' defined : [ 2123 2152 3386 ]
Detection:
  Search-Method = AC-Full-Q
  Split Any/Any group = enabled
  Search-Method-Optimizations = enabled
  Maximum pattern length = 20
Tagged Packet Limit: 256
Loading dynamic engine C:\Snort\lib\snort_dynamicengine\sf_engine.dll... done
Loading all dynamic preprocessor libs from C:\Snort\lib\snort_dynamicpreprocessor\...
Loading dynamic preprocessor library C:\Snort\lib\snort_dynamicpreprocessor\sf_dce2.dll... done
Loading dynamic preprocessor library C:\Snort\lib\snort_dynamicpreprocessor\sf_dnp3.dll... done
Loading dynamic preprocessor library C:\Snort\lib\snort_dynamicpreprocessor\sf_dns.dll... done
Loading dynamic preprocessor library C:\Snort\lib\snort_dynamicpreprocessor\sf_ftptelnet.dll... done
Loading dynamic preprocessor library C:\Snort\lib\snort_dynamicpreprocessor\sf_ftp.dll... done
Loading dynamic preprocessor library C:\Snort\lib\snort_dynamicpreprocessor\sf_imap.dll... done
Loading dynamic preprocessor library C:\Snort\lib\snort_dynamicpreprocessor\sf_modbus.dll... done
Loading dynamic preprocessor library C:\Snort\lib\snort_dynamicpreprocessor\sf_pop3.dll... done
Loading dynamic preprocessor library C:\Snort\lib\snort_dynamicpreprocessor\sf_reputation.dll... done
Loading dynamic preprocessor library C:\Snort\lib\snort_dynamicpreprocessor\sf_sdf.dll... done
Loading dynamic preprocessor library C:\Snort\lib\snort_dynamicpreprocessor\sf_sip.dll... done
Loading dynamic preprocessor library C:\Snort\lib\snort_dynamicpreprocessor\sf_smp.dll... done
Loading dynamic preprocessor library C:\Snort\lib\snort_dynamicpreprocessor\sf_ssh.dll... done
Loading dynamic preprocessor library C:\Snort\lib\snort_dynamicpreprocessor\sf_ssl.dll... done
Finished loading all dynamic preprocessor libs from C:\Snort\lib\snort_dynamicpreprocessor/
Log directory = C:\Snort\log
WARNING: ip4 normalizations disabled because not inline.
WARNING: tcp normalizations disabled because not inline.
WARNING: icmp4 normalizations disabled because not inline.
WARNING: ip6 normalizations disabled because not inline.
WARNING: icmp6 normalizations disabled because not inline.
Frag3 global config:
  Max Fragments: 65536
  Fragment memory cap: 4194304 bytes
Frag3 engine config:
  Bound Address: default
  Target-based policy: WINDOWS
  Fragment timeout: 180 seconds
  Fragment min_ttl: 1
  Fragment Anomalies: Alert
  Overlap Limit: 10
  Min fragment length: 100
  Max Expected Streams: 768
Stream global config:
  Track TCP sessions: ACTIVE
  Max TCP sessions: 262144
  TCP cache pruning timeout: 30 seconds
  TCP cache nominal timeout: 3600 seconds
  Memcap (for reassembly packet storage): 8388608
  Track UDP sessions: ACTIVE
  Max UDP sessions: 131072
  UDP cache pruning timeout: 30 seconds
  UDP cache nominal timeout: 180 seconds
  Track ICMP sessions: INACTIVE
  Track IP sessions: INACTIVE
  Log info if session memory consumption exceeds 1048576
  Send up to 2 active responses
  Wait at least 5 seconds between responses
  Protocol Aware Flushing: ACTIVE
  Maximum Flush Point: 16000
Stream TCP Policy config:
  Bound Address: default
  Reassembly Policy: WINDOWS
  Timeout: 180 seconds
  Limit on TCP Overlaps: 10
  Maximum number of bytes to queue per session: 1048576
  Maximum number of segs to queue per session: 2621
  Options:
    Require 3-Way Handshake: YES
    3-Way Handshake Timeout: 180
    Detect Anomalies: YES
```

POC'S

```
Select Administrator: Command Prompt
Experimental DNS RR Types Alert: INACTIVE
Ports: 53
SSLUP config:
Encrypted packets: not inspected
Ports:
443 465 563 636 989
992 993 994 995 7801
7802 7900 7901 7902 7903
7904 7905 7906 7907 7908
7909 7910 7911 7912 7913
7914 7915 7916 7917 7918
7919 7920
Server side data is trusted
Maximum SSL Heartbeat length: 0
Sensitive Data preprocessor config:
Global Alert Threshold: 25
Masked output: DISABLED
SIP config:
Max number of sessions: 40000
Max number of dialogs in a session: 4 (Default)
Status: ENABLED
Ignore media channel: DISABLED
Max URI length: 512
Max Call ID length: 80
Max Request name length: 20 (Default)
Max From length: 256 (Default)
Max To length: 256 (Default)
Max Via length: 1024 (Default)
Max Contact length: 512
Max Content length: 2048
Ports:
5060 5061 5060
Methods:
invite cancel ack bye register options refer subscribe update join info message notify benotify do qauth sprack publish service unsubscribe prack
IMAP Config:
Ports: 143
IMAP Maxcap: 838860
MIME Max Mem: 838860
Base64 Decoding: Enabled
Base64 Decoding Depth: Unlimited
Quoted-Printable Decoding: Enabled
Quoted-Printable Decoding Depth: Unlimited
Unix-to-Unix Decoding: Enabled
Unix-to-Unix Decoding Depth: Unlimited
Non-Encoded MIME attachment Extraction: Enabled
Non-Encoded MIME attachment Extraction Depth: Unlimited
POP Config:
Ports: 110
POP Maxcap: 838860
```

```
Select Administrator: Command Prompt
11808 Option Chains linked into 331 Chain Headers
-----[Rule Port Counts]-----
|      tcp      udp      icmp      ip
| src  3831      23         0         0
| dst  6442      77         0         0
| any   712       4         3         0
| nc    452       0         0         0
| s+d   4         2         0         0
-----[detection-filter-config]-----
| memory-cap : 1048576 bytes
-----[detection-filter-rules]-----
-----[rate-filter-config]-----
| memory-cap : 1048576 bytes
-----[rate-filter-rules]-----
| none
-----[event-filter-config]-----
| memory-cap : 1048576 bytes
-----[event-filter-global]-----
| none
-----[event-filter-local]-----
| none
-----[suppression]-----
| none
-----
Rule application order: pass->drop->sdrop->reject->alert->log
Verifying Preprocessor Configurations!
WARNING: flowbits key 'file.htm' is set but not ever checked.
WARNING: flowbits key 'tntftp' is set but not ever checked.
WARNING: flowbits key 'file.rpt' is set but not ever checked.
WARNING: flowbits key 'honet.4' is set but not ever checked.
WARNING: flowbits key 'file.pictmo' is set but not ever checked.
WARNING: flowbits key 'OptixPROv1.32download_detection' is set but not ever checked.
WARNING: flowbits key 'file.vap' is set but not ever checked.
WARNING: flowbits key 'file.dir' is set but not ever checked.
WARNING: flowbits key 'file.pcx' is set but not ever checked.
WARNING: flowbits key 'file.t23' is set but not ever checked.
WARNING: flowbits key 'AdvancedSpy_detection' is set but not ever checked.
WARNING: flowbits key 'file.screensaver' is set but not ever checked.
WARNING: flowbits key 'file.jpg' is set but not ever checked.
WARNING: flowbits key 'file.webm' is set but not ever checked.
WARNING: flowbits key 'file.r2g' is set but not ever checked.
WARNING: flowbits key 'file.pls' is set but not ever checked.
```


POC'S

